

OA Questions

1) SMALLEST GOOD BASE

Given an integer n represented as a string, return *the smallest **good base** of n* .

We call $k \geq 2$ a **good base** of n , if all digits of n base k are 1's.

Example 1:

Input: $n = "13"$

Output: $"3"$

Explanation: 13 base 3 is 111.

Example 2:

Input: $n = "4681"$

Output: $"8"$

Explanation: 4681 base 8 is 11111.

Example 3:

Input: $n = "10000000000000000000"$

Output: $"9999999999999999999"$

Explanation: 10000000000000000000 base 9999999999999999999 is 11.

Constraints:

- n is an integer in the range $[3, 10^{18}]$.
- n does not contain any leading zeros.

Sol). class Solution {

public:

string smallestGoodBase(string n) {

```

const long num = stol(n);

for (int m = log2(num); m >= 2; --m) {
    const int k = pow(num, 1.0 / m);
    long sum = 1;
    long prod = 1;
    for (int i = 0; i < m; ++i) {
        prod *= k;
        sum += prod;
    }
    if (sum == num)
        return to_string(k);
}

return to_string(num - 1);
}
};

```

2). COUNT GOOD TRIPLETS

Given an array of integers `arr`, and three integers `a`, `b` and `c`. You need to find the number of good triplets.

A triplet $(arr[i], arr[j], arr[k])$ is **good** if the following conditions are true:

- $0 \leq i < j < k < arr.length$
- $|arr[i] - arr[j]| \leq a$
- $|arr[j] - arr[k]| \leq b$
- $|arr[i] - arr[k]| \leq c$

Where $|x|$ denotes the absolute value of x .

Return *the number of good triplets*.

Example 1:

Input: arr = [3,0,1,1,9,7], a = 7, b = 2, c = 3

Output: 4

Explanation: There are 4 good triplets: [(3,0,1), (3,0,1), (3,1,1), (0,1,1)].

Example 2:

Input: arr = [1,1,2,2,3], a = 0, b = 0, c = 1

Output: 0

Explanation: No triplet satisfies all conditions.

Constraints:

- $3 \leq \text{arr.length} \leq 100$
- $0 \leq \text{arr}[i] \leq 1000$
- $0 \leq a, b, c \leq 1000$

Sol). class Solution:

```
def countGoodTriplets(self, arr: List[int], a: int, b: int, c: int) -> int:
    n = len(arr)
    count = 0
    for i in range(n):
        for j in range(i+1, n):
            for k in range(j+1, n):
                if(abs(arr[i] - arr[j]) <= a and abs(arr[j] - arr[k]) <= b and abs(arr[i] -
arr[k]) <= c):
                    count += 1
    return count
```

3)Count GOOD MEALS

A **good meal** is a meal that contains **exactly two different food items** with a sum of deliciousness equal to a power of two.

You can pick **any** two different foods to make a good meal.

Given an array of integers deliciousness where deliciousness[i] is the deliciousness of the i^{th} item of food, return *the number of different **good meals** you can make from this list modulo $10^9 + 7$.*

Note that items with different indices are considered different even if they have the same deliciousness value.

Example 1:

Input: deliciousness = [1,3,5,7,9]

Output: 4

Explanation: The good meals are (1,3), (1,7), (3,5) and, (7,9).

Their respective sums are 4, 8, 8, and 16, all of which are powers of 2.

Example 2:

Input: deliciousness = [1,1,1,3,3,3,7]

Output: 15

Explanation: The good meals are (1,1) with 3 ways, (1,3) with 9 ways, and (1,7) with 3 ways.

Constraints:

- $1 \leq \text{deliciousness.length} \leq 10^5$
- $0 \leq \text{deliciousness}[i] \leq 2^{20}$

Sol)

```
class Solution {
```

```
    int mod = 1000000007;
```

```
    public int countPairs(int[] arr) {
```

```
        Map<Integer, Integer> map = new HashMap<>();
```

```

int n = arr.length;
long res = 0;
for (int num : arr) {
    int power = 1;
    for (int i = 0; i < 22; i++) {
        if (map.containsKey(power - num)) {
            res += map.get(power - num);
            res %= mod;
        }
        power *= 2;
    }
    map.put(num, map.getDefault(num, 0) + 1);
}
return (int) res;
}
}

```

4)COUNT GOOD NUMBERS

A digit string is **good** if the digits (**0-indexed**) at **even** indices are **even** and the digits at **odd** indices are **prime** (2, 3, 5, or 7).

- For example, "2582" is good because the digits (2 and 8) at even positions are even and the digits (5 and 2) at odd positions are prime. However, "3245" is **not** good because 3 is at an even index but is not even.

Given an integer n , return *the **total** number of good digit strings of length n .* Since the answer may be large, **return it modulo** $10^9 + 7$.

A **digit string** is a string consisting of digits 0 through 9 that may contain leading zeros.

Example 1:**Input:** n = 1**Output:** 5**Explanation:** The good numbers of length 1 are "0", "2", "4", "6", "8".**Example 2:****Input:** n = 4**Output:** 400**Example 3:****Input:** n = 50**Output:** 564908303**Constraints:**

- $1 \leq n \leq 10^{15}$

Sol.

class Solution {

long mod = 1000000007;

public int countGoodNumbers(long n) {

return (int) ((quickmul(5, (n + 1) / 2) * quickmul(4, n / 2)) % mod);

}

// use fast exponentiation to calculate $x^y \% \text{mod}$

public long quickmul(int x, long y) {

long ret = 1;

long mul = x;

```

while (y > 0) {
    if (y % 2 == 1) {
        ret = (ret * mul) % mod;
    }
    mul = (mul * mul) % mod;
    y /= 2;
}

return ret;
}
}

```

5) GOOD Subsequence Queries

You are given an integer array `nums` of length `n` and an integer `p`.

A **non-empty subsequence** of `nums` is called **good** if:

- Its length is **strictly less** than `n`.
- The **greatest common divisor (GCD)** of its elements is **exactly** `p`.

You are also given a 2D integer array `queries` of length `q`, where each `queries[i] = [indi, vali]` indicates that you should update `nums[indi]` to `vali`.

After each query, determine whether there exists **any good subsequence** in the current array.

Return the **number** of queries for which a **good subsequence** exists.

The term `gcd(a, b)` denotes the **greatest common divisor** of `a` and `b`.

Example 1:

Input: `nums = [4,8,12,16]`, `p = 2`, `queries = [[0,3],[2,6]]`

Output: 1

Explanation:

i	[ind _i , val _i]	Operation	Updated nums	Any good Subsequence
0	[0, 3]	Update nums[0] to 3	[3, 8, 12, 16]	No, as no subsequence has GCD exactly p = 2
1	[2, 6]	Update nums[2] to 6	[3, 8, 6, 16]	Yes, subsequence [8, 6] has GCD exactly p = 2

Thus, the answer is 1.

Example 2:

Input: nums = [4,5,7,8], p = 3, queries = [[0,6],[1,9],[2,3]]

Output: 2

Explanation:

i	[ind _i , val _i]	Operation	Updated nums	Any good Subsequence
0	[0, 6]	Update nums[0] to 6	[6, 5, 7, 8]	No, as no subsequence has GCD exactly p = 3
1	[1, 9]	Update nums[1] to 9	[6, 9, 7, 8]	Yes, subsequence [6, 9] has GCD exactly p = 3
2	[2, 3]	Update nums[2] to 3	[6, 9, 3, 8]	Yes, subsequence [6, 9, 3] has GCD exactly p = 3

Thus, the answer is 2.

Example 3:

Input: nums = [5,7,9], p = 2, queries = [[1,4],[2,8]]

Output: 0

Explanation:

i	[ind _i , val _i]	Operation	Updated nums	Any good Subsequence
0	[1, 4]	Update nums[1] to 4	[5, 4, 9]	No, as no subsequence has GCD exactly p = 2

i	[ind _i , val _i]	Operation	Updated nums	Any good Subsequence
1	[2, 8]	Update nums[2] to 8	[5, 4, 8]	No, as no subsequence has GCD exactly p = 2

Thus, the answer is 0.

Constraints:

- $2 \leq n == \text{nums.length} \leq 5 * 10^4$
- $1 \leq \text{nums}[i] \leq 5 * 10^4$
- $1 \leq \text{queries.length} \leq 5 * 10^4$
- $\text{queries}[i] = [\text{ind}_i, \text{val}_i]$
- $1 \leq \text{val}_i, p \leq 5 * 10^4$
- $0 \leq \text{ind}_i \leq n - 1$

Sol)

```
class Solution {
```

```
public:
```

```
    int n, P;
```

```
    vector<int> seg, arr;
```

```
    int gcdv(int a,int b){
```

```
        if(!a) return b;
```

```
        if(!b) return a;
```

```
        return gcd(a,b);
```

```
    }
```

```
    void build(int i,int l,int r){
```

```
        if(l==r){
```

```

        seg[i] = (arr[l]%P==0 ? arr[l]/P : 0);
        return;
    }

    int m=(l+r)/2;

    build(i*2,l,m);
    build(i*2+1,m+1,r);

    seg[i]=gcdv(seg[i*2],seg[i*2+1]);
}

void update(int i,int l,int r,int idx,int val){
    if(l==r){
        seg[i]=(val%P==0 ? val/P : 0);
        return;
    }

    int m=(l+r)/2;

    if(idx<=m) update(i*2,l,m,idx,val);
    else update(i*2+1,m+1,r,idx,val);

    seg[i]=gcdv(seg[i*2],seg[i*2+1]);
}

```

```

bool canExcludeOne(int i,int l,int r,int ext){
    if(l==r) return ext==1;

    int m=(l+r)/2;

    int left = gcdv(ext, seg[i*2+1]);
    int right = gcdv(ext, seg[i*2]);

    if(left==1 || right==1) return true;

    return canExcludeOne(i*2,l,m,left) ||
           canExcludeOne(i*2+1,m+1,r,right);
}

```

```

int countGoodSubseq(vector<int>& nums, int p, vector<vector<int>>&
queries) {

```

```

    arr = nums;

```

```

    P = p;

```

```

    n = arr.size();

```

```

    seg.assign(4*n,0);

```

```

    int cnt = 0;

```

```

    for(int x:arr) if(x%P==0) cnt++;

```

```

    build(1,0,n-1);

```

```

int ans=0;

for(auto &q:queries){
    int idx=q[0], val=q[1];

    if(arr[idx]%P==0) cnt--;
    if(val%P==0) cnt++;

    arr[idx]=val;

    update(1,0,n-1,idx,val);

    if(seg[1]==1){
        if(cnt<n) ans++;
        else if(canExcludeOne(1,0,n-1,0)) ans++;
    }
}

return ans;
}
};

```

6) CHECK GOOD INTEGER

You are given a positive integer n .

Let digitSum be the sum of the digits of n , and let squareSum be the sum of the squares of the digits of n .

An integer is called **good** if $\text{squareSum} - \text{digitSum} \geq 50$.

Return true if n is good. Otherwise, return false.

Example 1:

Input: n = 1000

Output: false

Explanation:

- The digits of 1000 are 1, 0, 0, and 0.
- The digitSum is $1 + 0 + 0 + 0 = 1$.
- The squareSum is $1^2 + 0^2 + 0^2 + 0^2 = 1$.
- The squareSum - digitSum is $1 - 1 = 0$. As 0 is not greater than or equal to 50, the output is false.

Example 2:

Input: n = 19

Output: true

Explanation:

- The digits of 19 are 1 and 9.
- The digitSum is $1 + 9 = 10$.
- The squareSum is $1^2 + 9^2 = 1 + 81 = 82$.
- The squareSum - digitSum is $82 - 10 = 72$. As 72 is greater than or equal to 50, the output is true.

Constraints:

- $1 \leq n \leq 10^9$

Sol). class Solution {

public:

bool checkGoodInteger(int n) {

```

long long sum=0,sum2=0;
while(n>0){
    int dig=n%10;
    sum+=dig;
    sum2+=(dig*dig);
    n/=10;
}
return sum2-sum>=50;
}
};

```

7) COUNT GOOD Subarrays

You are given an integer array nums.

A **subarray** is called **good** if the **bitwise OR** of all its elements is equal to **at least one** element present in that subarray.

Return the number of good subarrays in nums.

Here, the bitwise OR of two integers a and b is denoted by $a \mid b$.

Example 1:

Input: nums = [4,2,3]

Output: 4

Explanation:

The subarrays of nums are:

Subarray	Bitwise OR	Present in Subarray
[4]	$4 = 4$	Yes
[2]	$2 = 2$	Yes
[3]	$3 = 3$	Yes

[4, 2]	$4 \mid 2 = 6$	No
[2, 3]	$2 \mid 3 = 3$	Yes
[4, 2, 3]	$4 \mid 2 \mid 3 = 7$	No

Thus, the good subarrays of nums are [4], [2], [3] and [2, 3]. Thus, the answer is 4.

Example 2:

Input: nums = [1,3,1]

Output: 6

Explanation:

Any subarray of nums containing 3 has bitwise OR equal to 3, and subarrays containing only 1 have bitwise OR equal to 1.

In both cases, the result is present in the subarray, so all subarrays are good, and the answer is 6.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $0 \leq \text{nums}[i] \leq 10^9$

Sol)

```
class Solution {
```

```
public:
```

```
    long long countGoodSubarrays(vector<int>& nums) {
```

```
        int n = nums.size();
```

```
        vector<int> nder(n, n), ndel(n, -1);
```

```
        stack<int> st;
```

```
        for(int i=0; i<n; i++){
```

```
            // nums[i] is not submask of nums[st.top()]
```

```

while(!st.empty() && ((nums[st.top()] | nums[i]) > nums[st.top()])) {
    nder[st.top()] = i;
    st.pop();
}

st.push(i);
}

while(!st.empty()) st.pop();
for(int i=n-1; i>=0; i--){
    while(!st.empty() && ( ((nums[st.top()] | nums[i]) > nums[st.top()]) ||
(nums[i] == nums[st.top()]) )) ) {
        ndel[st.top()] = i;
        st.pop();
    }

    st.push(i);
}

long long ans = 0;
for(int i=0; i<n; i++){
    long long r = nder[i] -i;
    long long l = i-ndel[i];
    ans += l *r;
}

```



```
        return ans;
    }
};
```